

Nacos主题分享

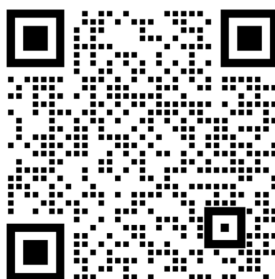
自我介绍：

- 赵延
- 基础架构，服务治理组
- Java爱好者
- 开源爱好者（Apache Dubbo Committer, Alibaba Nacos Committer）
- Github ID @horizonzy
- 技术撕逼群



PHP是世界上最好的语言

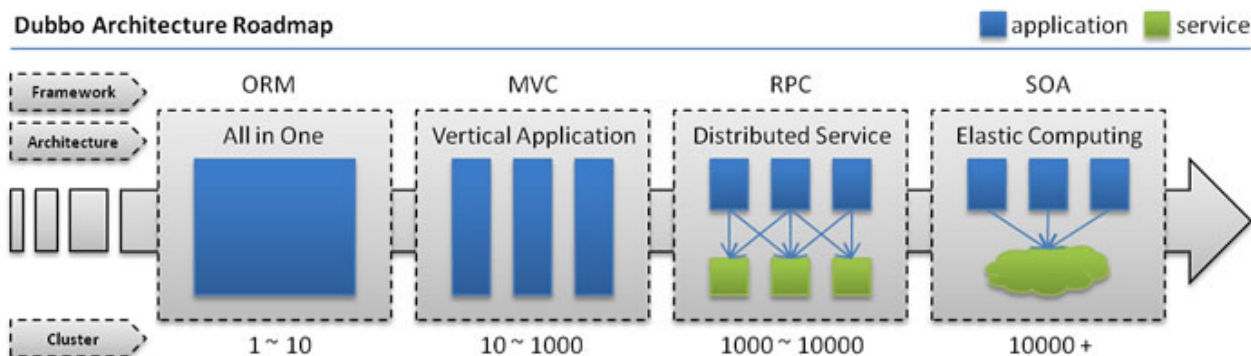
此群是企业内部群聊，仅企业成员可扫码加入



该二维码 4月12日前有效

 企业微信

一、微服务架构的由来



单一架构应用

当网站流量很小时，只需一个应用，将所有功能都部署在一起，以减少部署节点和成本。此时，用于简化增删改查工作量的数据访问框架(ORM)是关键。

垂直应用架构

当访问量逐渐增大，单一应用增加机器带来的加速度越来越小，提升效率的方法之一是将应用拆成互不相干的几个应用，以提升效率。此时，用于加速前端页面开发的Web框架(MVC)是关键。

分布式服务架构

当垂直应用越来越多，应用之间交互不可避免，将核心业务抽取出来，作为独立的服务，逐渐形成稳定的服务中心，使前端应用能更快速的响应多变的市场需求。此时，用于提高业务复用及整合的分布式服务框架(RPC)是关键。

流动计算架构

当服务越来越多，容量的评估，小服务资源的浪费等问题逐渐显现，此时需增加一个调度中心基于访问压力实时管理集群容量，提高集群利用率。此时，用于提高机器利用率的资源调度和治理中心(SOA)是关键。

二、微服务架构下服务发现演进

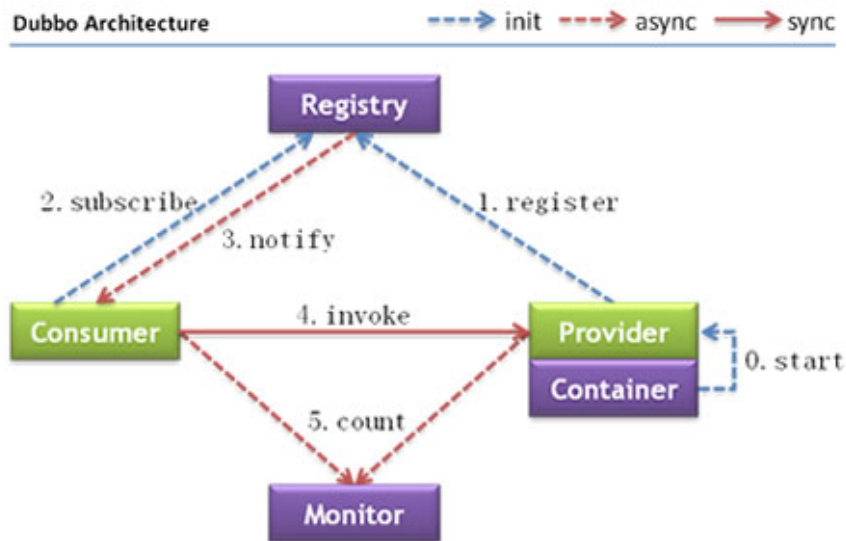
配置URL

在大规模服务化之前，应用可能只是通过 RMI 或 Hessian 等工具，简单的暴露和引用远程服务，通过配置服务的URL地址进行调用，通过 F5 等硬件进行负载均衡

引入注册中心

当服务越来越多时，服务 URL 配置管理变得非常困难，F5 硬件负载均衡器的单点压力也越来越大。此时需要一个服务注册中心，动态地注册和发现服务，使服务的位置透明。并通过在消费方获取服务提供方地址列表，实现软负载均衡和 Failover，降低对 F5 硬件负载均衡器的依赖，也能减少部分成本。

注册中心模式下的架构



节点	角色说明
Provider	暴露服务的服务提供方
Consumer	调用远程服务的服务消费方
Registry	服务注册与发现的注册中心
Monitor	统计服务的调用次数和调用时间的监控中心
Container	服务运行容器

三、CAP理论

著名的CAP理论，指的是在一个分布式系统中，[一致性](#)（Consistency）、[可用性](#)（Availability）、[分区容错性](#)（Partition tolerance）。CAP 原则指的是，这三个[要素](#)最多只能同时实现两点，不可能三者兼顾。

一致性：就是数据保持一致，在分布式系统中，可以理解为多个节点中数据的值是一致的。

可用性：在某个考察时间，系统能够正常运行的概率或时间占有率期望值

分区容错性：是指系统能够容忍节点之间的网络通信的故障

四、注册中心要求

1. 提供注册订阅机制

提供者能够将自己的信息注册到注册中心，消费者只需要订阅该信息，就能在提供者上下线时进行感知。

2. 宕机自动摘除

提供者在宕机后，需要在一段时间内将自己从注册中心的数据删除，避免消费者还会调用已经宕机的机器。

3. 可横向扩展

在服务越来越多，对应的机器实例也越来越多的情况下，注册到注册中心的数据会越来越多，注册中心需要能够很好地进行扩展，增大整个集群的承载能力。

4. 数据一致性

因为注册中心往往是集群部署的，客户端可能会连接到不同的节点，所以在集群的各个节点间的数据需要保持一致。

5. 多语言sdk支持

对于使用者来说，在公司内部存在多种语言的项目，在进行服务注册订阅时，最好能够具备各个不同的语言的SDK。

五、CP模型的注册中心Zookeeper

在dubbo推出的那个时间点，使用zookeeper作为注册中心是大多数开发者的首选，以至于现在可能少部分开发者提到zk就认为是注册中心，可见Zookeeper在服务发现的领域也是有着举足轻重的贡献。

Zookeeper对照第四节的内容中的实现：

提供注册订阅机制： Zookeeper客户端可以通过API创建某个节点来实现注册机制，也可以通过服务端提供的Watcher机制，监听某一个节点下的节点变更事件，（例如创建，删除，修改）来对节点进行监听，当节点进行变更时，客户端能够立即进行感知

宕机自动摘除： Zookeeper中的节点存在EPHEMERAL类型，也就是我们常说的临时节点。该类型的节点会和session绑定，只要session过期（一般为40s），该节点就会被Zookeeper服务端进行删除。而维护session一直存在是通过Zookeeper客户端定时给服务端发送心跳进行续约，只要对应的应用实例宕机，Zookeeper客户端就无法继续发送心跳，临时节点随即会被删除

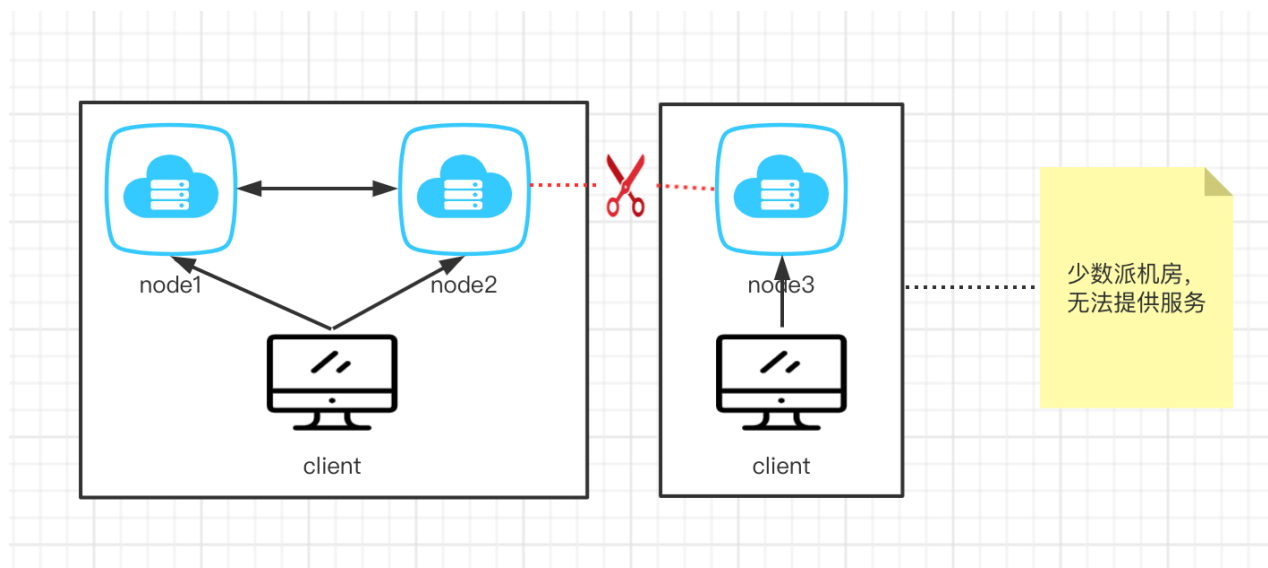
可横向扩展： Zookeeper中支持集群搭建，也可随时增加集群节点数，因为内置的ZAB协议，一般推荐节点数为奇数个节点。不过也是由于ZAB协议的存在（写请求必须由leader处理），导致Zookeeper的写性能存在瓶颈。不过读请求是可以通过增加节点个数来分摊整体集群的压力

数据一致性： Zookeeper中内置了ZAB协议来进行一致性的保证，写请求由leader进行处理，每一次的写请求会生成对应的zxid，然后向所有follower节点发起一次提议，只要有半数的节点返回了ack，leader节点就会将本次请求commit，并将commit消息广播给其他follower。读数据时会将客户端当前的zxid携带在请求中，服务端发现客户端的zxid大于自己的zxid时，会拒绝该请求，随即客户端会通过负

载请求到另一个节点，这样客户端能够保证当前session的读请求一定能读取到最新的数据。

多语言SDK支持： Zookeeper支持多种客户端，生态维护非常不错

弊端： 由于Zookeeper内部使用ZAB协议来保证一致性，因为ZAB协议是基于CP模型进行设计的，因此当某个机房和其他机房发生网络分区，少数派机房的Zookeeper就无法提供服务了，少数派机房的客户端就无法感知到服务实例的上下线，会损失一部分流量。并且写请求只能由leader来进行处理，在服务同时大量上下线时，Zookeeper服务端的压力无法进行分摊。



总结： Zookeeper具备注册中心的功能，但由于其CP模型的设计，在发生网络分区时，会导致一部分流量损失，这其实是不可接受的。在服务发现的场景，可用性其实比一致性更加重要，只需要保证最终一致即可。不过由于早期并没有合适的AP模型的注册中心存在，Zookeeper依然陪伴大部分公司走过了最开始的发展阶段。

六、AP模型的注册中心Nacos

正是由于CP模型在服务发现场景下存在的弊端，我们需要一个AP模型的服务发现组件，来避免发生网络分区后带来的问题。于是，Nacos诞生了。

官方介绍： 英文全称Dynamic Naming and Configuration Service，Na为naming/nameServer即注册中心，co为configuration即配置中心，service是指该注册/配置中心都是以服务为核心。服务在nacos是一等公民。

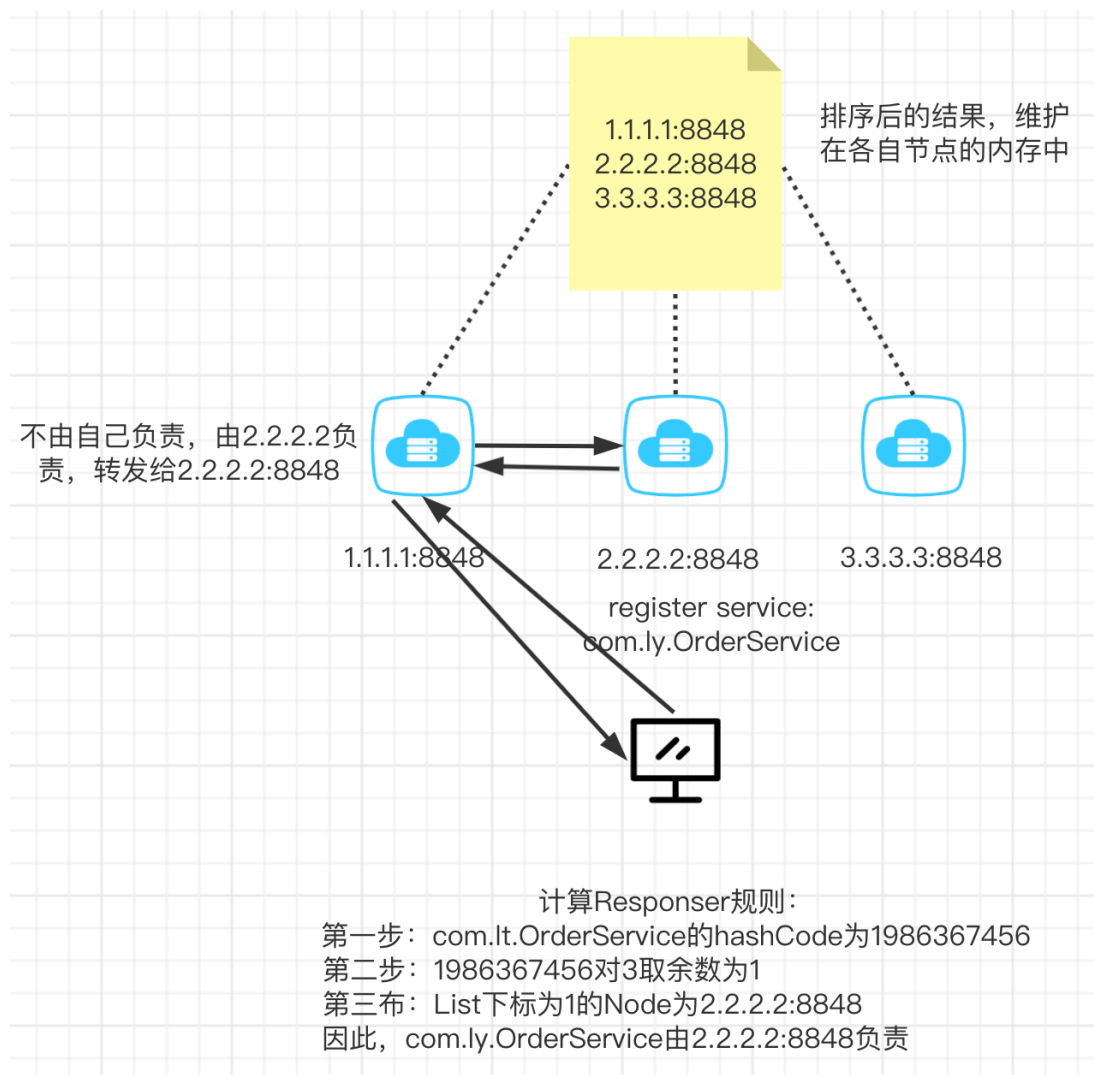
这次我们主要分享服务发现的Naming模块，其由Nacos中的AP模型由内置的Distro协议实现。

Distro协议中主要概念：

1. **Member：** 在Nacos-Server启动时，会在cluster.conf中配置整个集群列表，其作用是让每个节点都知道集群中的所有节点，该列表中的每一个节点都抽象成一个Member

2. **MemberInfoReportTask**: 在Nacos-Server启动成功后, 会定时给除自己之外的其他Member进行通信, 检测其他节点是否还存活。如果通信失败, 会将该Member状态置为不健康的, 如果后续和该节点重新通信成功, 会将该节点的状态置为健康, 该Task与Responser的计算密切相关
3. **Responser**: 对于每一个服务 (比如: `com.ly.OrderService`) 来说, 在Nacos-Server集群中都会有一个专门的节点来负责。比如集群中有三个健康节点, 这三个节点的IP:Port就是组成一个长度为3的List<String>, 对三个节点的IP:Port组成的addressList进行排序, 这样在每一个节点中, addressList的顺序都是一致的。这时 `com.ly.OrderService` 服务注册上来, 会根据服务名计算对应的hash值, 然后对集群的节点数取余获得下标, 从addressList中获取对应的IP:Port, 这时这个IP:Port对应的节点就是该服务的Responser, 负责该服务的健康检查, 数据同步, 心跳维持, 服务注册。如果客户端服务注册请求到了某个节点, 但是本次注册的服务不是由该节点负责, 会将该请求重定向到responser的节点去进行处理。

注意: 这里的addressList是健康节点, 一旦某个节点宕机或者网络发生故障, 该节点会从addressList中移除, Service对应的Responser会发生变化。



4. **HealthChecker**: 对于每一个服务, 都有一个HealthCheck去对其中的实例进行检测, 检测的原则就是该实例最近上报心跳的时间与当前时间的时间差是否超过阈值, 如果超过阈值, 需要将该实例从服务中摘除。HealthCheck在检测时, 也会去进行Responser的检查, 只有自己是当前服务的Responser, 才会去进行检测。
5. **DistroTask**: 由于Responser规则的存在, 对于某一个服务来说, 只会有一个node来进行负责, 那么其他的node是如何感知到非responser节点的服务数据的呢。DistroTask就是做数据同步的, 对当前自己持有的所有服务进行检测, 只要有是自己response的, 就把该服务的实例数据同步给其

他node。这里就有一个优化点，在同步数据时，并不是把该服务下所有的实例全部同步给其他节点，而是对该服务当前所有实例计算一个checksum值（减少传输的数据量，而且一般来说，实例变动是不频繁的）同步到其他节点。其他节点收到数据后，首先会检查同步过来的服务是否是由远端负责，如果是，比对自己节点中该服务的checksum值和远端的是否一致，如果不一致，请求远端节点获取最新的实例数据，再更新本地数据。

6. **LoadDataTask**：在节点刚启动时，会主动向其他节点拉取一次全量的数据，来让当前节点和整个集群中的数据快速保持一致。

Nacos对照第四节的内容中的实现：

提供注册订阅机制：客户端通过SDK提供的API可直接进行服务注册，将服务和对应的实例数据传到服务端，服务端进行保存。在订阅时，首先会在客户端开启的UDP端口，在订阅时将该端口传递到服务端。后续服务端对应的服务实例发生变化后，会直接通过对UDP端口发送数据，客户端收到数据后更新数据，并进行通知。同时，因为UDP的不稳定性，客户端也会定时去轮询服务端，检查服务的实例数据是否发生变化，如果发生变化，也会更新数据并进行通知。

宕机自动摘除：客户端在进行服务实例注册时，会在本地启动一个定时任务，每隔几秒钟就会向服务端发送一个心跳请求，服务端收到心跳请求后，更新实例的最后一次心跳时间。一旦客户端宕机，就不会继续向服务端发送心跳请求，Distro协议中的HealthChecker就会将该实例进行摘除。

可横向扩展：Nacos在启动时，就会监听cluster.conf的变化，去动态更新Member列表。一旦集群负载达到瓶颈，可以启动新的Nacos-Server，再把这台机器的地址添加到其他节点的cluster.conf文件中，其他节点就动态感知到了整个集群节点数的增加，对应的Service的Responser重新计算，因为是Hash对数量取余数，Service会均匀得摊分到各个节点中。而且由于Responser机制的存在，没有Leader的概念，写并不会向Zookeeper那种成为瓶颈。

数据一致性：因为Distro协议Responser机制的存在，不同的节点处理不同的Service，然后通过DistroTask来将数据进行同步。因为数据是通过同步来保持一致的，所以Nacos中的一致是最终一致。客户端在订阅数据时，只要是服务端的数据发生了变化，会通过UDP主动推给客户端，只要是服务端的数据是最终一致的，最后推给客户端的数据也是一致的。即使UDP推送丢包，客户端也会定时去轮训服务端的数据，保证客户端的数据和服务端中最终一致。

多语言SDK支持：Nacos目前支持多种语言的SDK，生态体系十分不错。

七、Nacos AP模型发生网络分区

Nacos对比Zookeeper，其AP协议在发生网络分区的情况下，最终效果是如何的呢？下面会带大家一起梳理一下。

服务端：三个节点，IP分别为1.1.1.1:8848（node1），2.2.2.2:8848（node2），3.3.3.3:8848（node3）。三个节点位于不同的机房。Health MemberList为[1.1.1.1:8848, 2.2.2.2:8848, 3.3.3.3:8848]

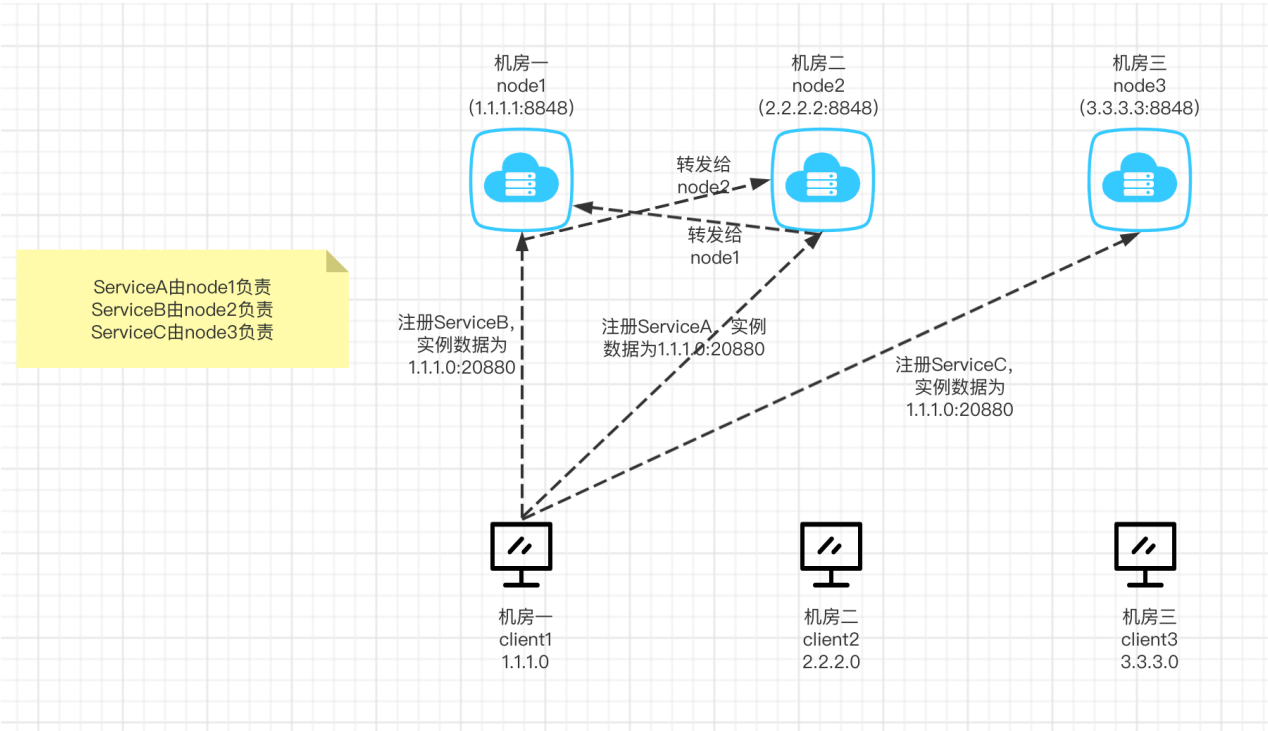
客户端：3个客户端，IP分别为1.1.1.0（client1），2.2.2.0（client2），3.3.3.0（client3）在上面的三个机房中各自一个客户端。需要注意的是，在客户端中必须要填所有的Nacos-Server的地址。不止是Nacos，像其他的分布式系统的客户端，一般都是需要填多个服务端地址，在客户端内会带有负载均衡算法，请求会分摊到不同的Nacos-Server。并且当某个Nacos-Server挂掉之后，客户端依旧能够请求其他的Nacos-Server。Nacos客户端目前是使用的Random Load Balance，请求某个节点失败后，自动请

求下一个节点，当客户端的所有地址都请求失败后，抛出异常。

注册的服务： ServiceA, ServiceB, ServiceC, 三个客户端都会将自己注册到这三个服务中，使用的端口为20880。ServiceA的Hash值为3，ServiceB的Hash为1，ServiceC的hash值为2。

阶段一：服务端启动完成，客户端启动

客户端的是随机Load Balance，因此3个注册请求刚好打在了3台不同的node上。根据Responder的算法，ServiceA由node1负责，ServiceB由node2负责，ServiceC由node3负责。



client1, client2, 和client3都注册实例完之后，结果如下：

node1:

服务名称	实例数据
ServiceA	[1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]

node2:

服务名称	实例数据
ServiceB	[1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]

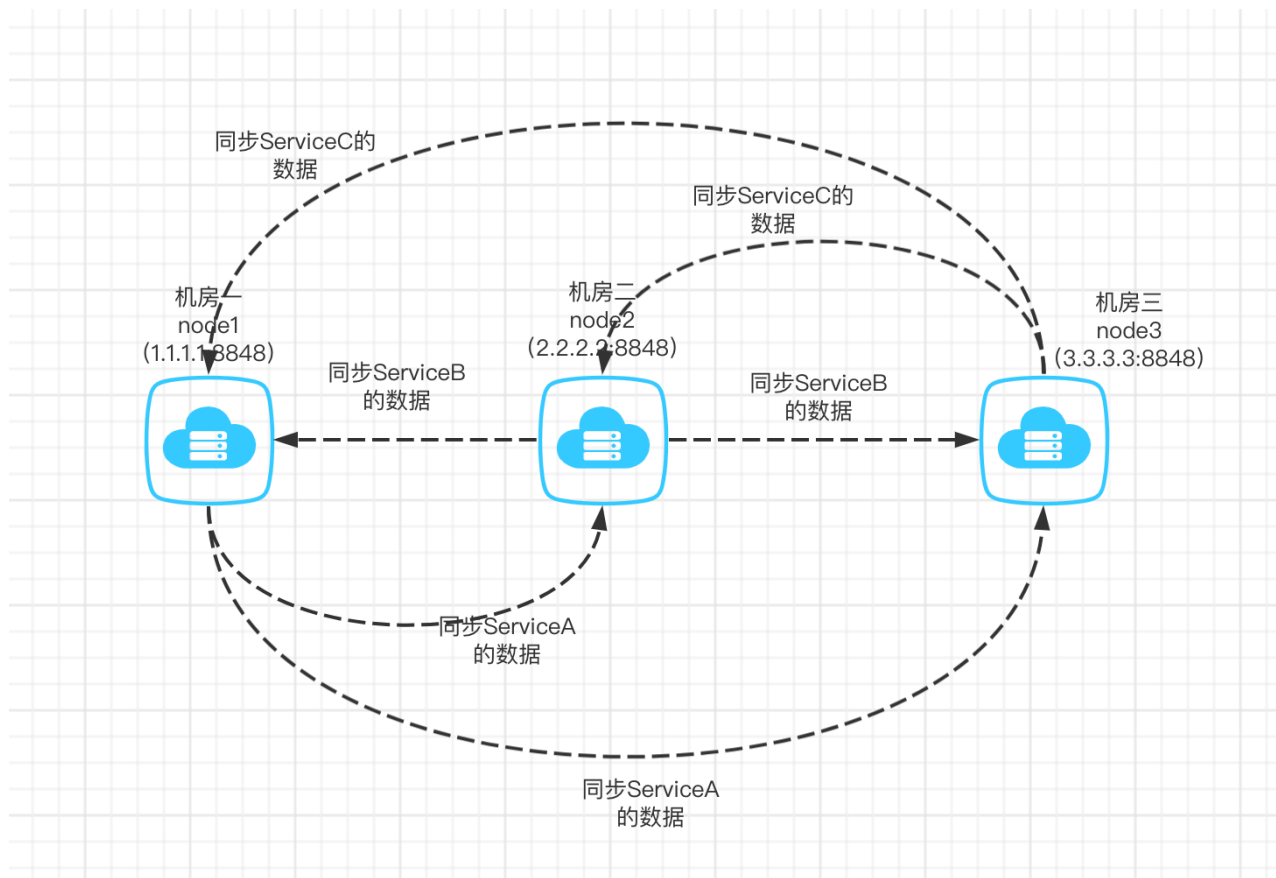
node3:

服务名称	实例数据
ServiceC	[1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]

在注册完之后，客户端只会定期去发送心跳，维系对应的实例。

阶段二：服务端开始同步数据

node1把自身负责的ServiceA同步给node2和node3，node2把自己负责的ServiceB同步给node1和node3，node3把自己负责的ServiceC同步给node1，node2。



服务端同步完数据后，结果如下：

node1:

服务名称	实例数据
ServiceA	[1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]
ServiceB	[1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]
ServiceC	[1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]

node2:

服务名称	实例数据
ServiceA	[1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]
ServiceB	[1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]
ServiceC	[1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]

node3:

服务名称	实例数据
ServiceA	[1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]
ServiceB	[1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]
ServiceC	[1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]

node1中存在ServiceA服务，持有实例 [1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]，存在ServiceB服务，持有实例 [1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]，存在ServiceC服务，持有实例 [1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]。

node2和node3同node1一样，持有所有服务的所有实例。

在这之后，不用担心node1把ServiceB和ServiceC重新同步给node2，node3。因为根据Responder原则，node1只会同步ServiceA。

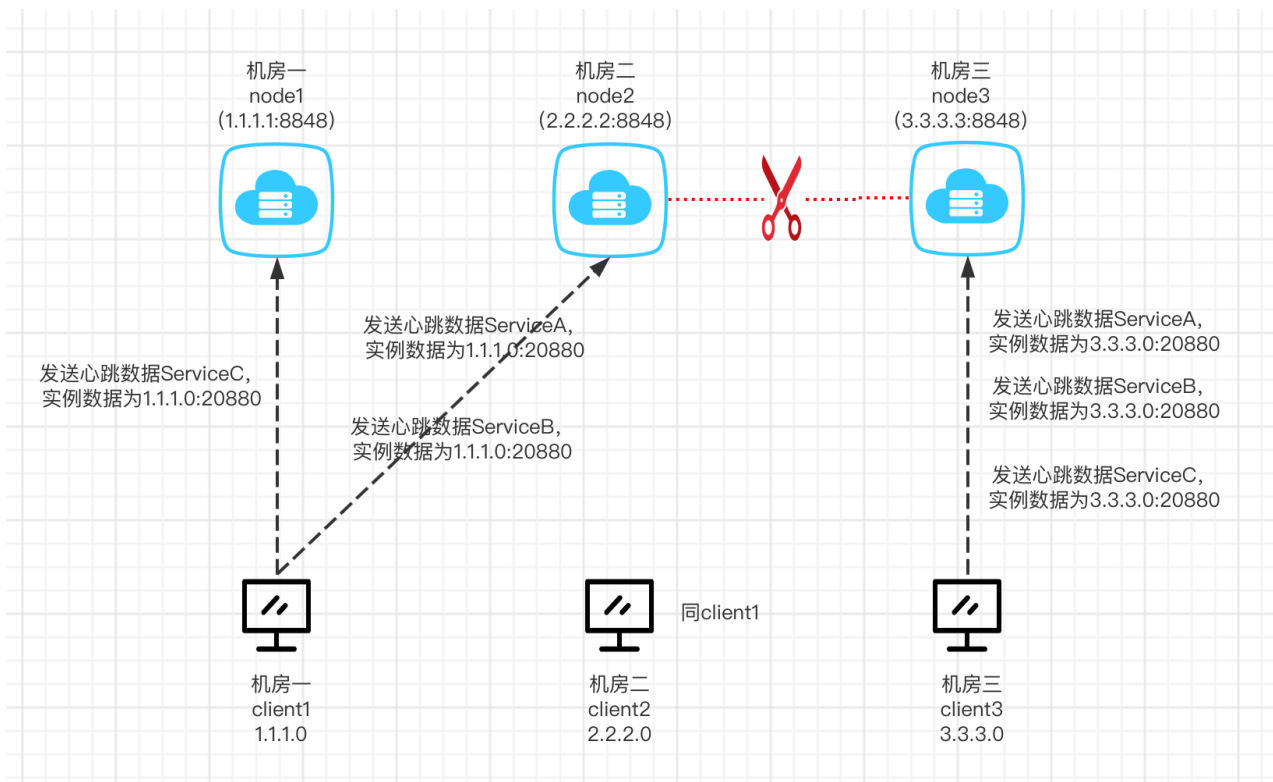
阶段三：机房三发生网络分区，和机房一和机房二的网络无法联通。

在服务端，由于node1，node2与node3的网络无法进行通信了，在node1和node2中，会把node3的状态设置为不健康。在node3中，会把node1和node2的状态设置为不健康。此时，对应的Responder规则需要重新进行计算。

在node1和node2中，Health MemberList为 [1.1.1.1:8848, 2.2.2.2:8848]，因此node1和node2达成共识，ServiceA由node2负责，ServiceB由node2负责，ServiceC由node1负责。

在node3中，Health Member List为 [3.3.3.3:8848]。ServiceA，ServiceB和ServiceC都由自己负责。

在客户端，由于网络分区。client1和client2只能请求到node1和node2，client3只能请求到node3。



最终会发生的结果：

客户端3无法继续上报心跳到node1和node2中。node1中负责ServiceC的HealthChecker会把ServiceC中的实例3.3.3.0:20880摘除。node2中负责ServiceA和ServiceB的HealthChecker会把ServiceA中和ServiceB中的实例3.3.3.0:20880摘除。

node1:

服务名称	实例数据
ServiceA	[1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]
ServiceB	[1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]
ServiceC	[1.1.1.0:20880, 2.2.2.0:20880]

node2:

服务名称	实例数据
ServiceA	[1.1.1.0:20880, 2.2.2.0:20880]
ServiceB	[1.1.1.0:20880, 2.2.2.0:20880]
ServiceC	[1.1.1.0:20880, 2.2.2.0:20880, 3.3.3.0:20880]

node1将自己负责的ServiceC的数据同步给node2，node2收到数据后，将自己ServiceC的数据更新和node1保持一致。

node2将自己负责的ServiceA，ServiceB同步给node1，node1收到数据后，将自己的ServiceA，ServiceB的数据更新和node2保持一致。

node1:

服务名称	实例数据
ServiceA	[1.1.1.0:20880, 2.2.2.0:20880]
ServiceB	[1.1.1.0:20880, 2.2.2.0:20880]
ServiceC	[1.1.1.0:20880, 2.2.2.0:20880]

node2:

服务名称	实例数据
ServiceA	[1.1.1.0:20880, 2.2.2.0:20880]
ServiceB	[1.1.1.0:20880, 2.2.2.0:20880]
ServiceC	[1.1.1.0:20880, 2.2.2.0:20880]

在node客户端1和客户端2也无法继续上报心跳到node3。node3也会把ServiceA中的[1.1.1.0:20880, 2.2.2.0:20880]摘除。ServiceB和ServiceC也是如此，

node3:

服务名称	实例数据
ServiceA	[3.3.3.0:20880]
ServiceB	[3.3.3.0:20880]
ServiceC	[3.3.3.0:20880]

总结： 网络分区发生之后，在各自的机房内，服务发现依然能够继续使用，没有少数派不可用的情况。在服务发现的场景下，确实是比Zookeeper更加适合。

八、我是如何成为Nacos Committer的

随着公司规模的发展，线上所承载的服务数量越来越多，一旦发生事故，对业务带来的影响是巨大的。在服务发现的场景，由于历史原因，使用的是zookeeper作为注册中心的场景，而由于其CP的模式在服务发现的场景下并不是很合适，在选新的注册中心时，决定使用nacos来作为zookeeper的替换组件，于是就开启了自己对于nacos的调研之路。最开始，就从源码开始看起，遇到觉得逻辑有问题的地方就会提相关的issue以及pr，社区的负责人@杨翊 很快就回复了,第一次参与开源的我十分激动。随后他详尽的给出了他对问题的看法以及建议，在他的建议下提交了自己的第一个pr并通过了，这极大地鼓舞了我参与开源的劲头，感谢@杨翊在我参与nacos社区中所提供的帮助。后续在对nacos熟悉了之后，开始对社区中的一些issue进行回复处理，解决一些小伙伴使用nacos中出现的一些问题，对一些代码进行了优化以及修复相关bug。在参与社区半年之后，社区负责人发了一封邀请邮件，同意后就正式成为Nacos Committer啦。

九、如何参与开源项目

1. 阅读源码，对整体项目结构有一定的了解
2. 关注社区相关的issue， `Good first issue` 或者 `Help wanted` 标签的，可以找其中一些比较简单开始入手
3. 在对整体项目比较熟悉了之后，自己思考项目中一些比较不错的功能，自己提feature和社区讨论，没问题的话就可以出设计文档进行开发了
4. 完善文档，一些项目在早期时，文档是不怎么完善的，可以在阅读代码之后，提交相关的文档到社区

十、更多

本次我们分享的主要是Nacos的Naming模块，如果对Config模块也感兴趣的话可以在下次再进行分享。以上内容有不明白的地方需要讨论的话，可以直接微信找我。